VibeStack

An Architecture-Aware AI Software Engineering Harness

Project Phase I Report submitted in partial fulfillment for the degree of
B.Tech in CSE(Artificial Intelligence)

Submitted by
Reuben Sabu V (Reg No: MUT23CA060)
Sidharth Ravi (Reg No: MUT23CA063)
Jino Jenz (Reg No: MUT23CA035)
Nikhil Jimmy (Reg No: MUT23CA055)

Muthoot Institute of Technology and Science Kochi (Autonomous)

Department of Artificial Intelligence and Data Science
(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026



DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

CERTIFICATE

This is to certify that the Project Phase I report entitled **"VibeStack: An Architecture-Aware AI Software Engineering Harness"**, submitted by Reuben Sabu V (MUT23CA060), Sidharth Ravi (MUT23CA063), Jino Jenz (MUT23CA035), Nikhil Jimmy (MUT23CA055) of Semester VII is a bonafide account of the work done by them under our supervision during the academic year 2026-27.

Guide	**Coordinator(s)**	**Head of the Department**
Nimmi M K	Dr. Parvathy Jyothi / Ms. Angel Varghese	Dr. Cerene Mariam Abraham
Assistant Professor	Asst. Professor	Associate Professor
Dept. of AI&DS	Dept. of AI&DS	Dept. of AI&DS

DECLARATION

We hereby declare that this submission is our own work and that, to the best of our knowledge and belief, it contains no material previously written by another person nor material which has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgement has been made in the text.

The overall similarity index as per the Turnitin plagiarism check is **XX%**.

Reuben Sabu V (Reg No: MUT23CA060)
Sidharth Ravi (Reg No: MUT23CA063)
Jino Jenz (Reg No: MUT23CA035)
Nikhil Jimmy (Reg No: MUT23CA055)

Place: Kochi
Date:

ACKNOWLEDGEMENTS

This project report is the result of our hard work wherein we have been helped and supported by several persons and institutions directly and indirectly. Now it is the time to acknowledge their valuable contributions.

We would like to extend our sincere gratitude to the Management of Muthoot Institute of Technology and Science for providing us with the necessary facilities, resources, and a conducive environment to carry out our project successfully.

We are deeply thankful to Dr. Neelakantan P. C., Principal, Muthoot Institute of Technology and Science, for his constant support, guidance, and encouragement throughout our academic journey. Our sincere gratitude to Dr. Cerene Mariam Abraham, Head of the Department during our course period, for her guidance and support. Her encouragement and leadership have greatly helped us in successfully completing our project.

We would like to express our heartfelt gratitude to our project guide, Nimmi M K, Assistant Professor, Department of Artificial Intelligence and Data Science, for the continuous encouragement, invaluable guidance, motivation, and enthusiasm shown throughout our project. We would also like to thank the project coordinators Dr. Parvathy Jyothi and Ms. Angel Varghese, for critically assessing our work and providing valuable suggestions.

Besides this, several people have knowingly and unknowingly helped us in the successful completion of this project. We express our sincere gratitude to all of them.

Abstract

AI coding assistants can now write hundreds of lines of code in seconds, but the code they hand back is rarely checked. For a multi-file backend this is a real problem: a wrong import, a missing foreign key or a forgotten permission check only shows up when someone tries to run the application, and finding the cause often takes longer than writing the code by hand. Most of the popular "app builder" tools are also built around JavaScript stacks, which leaves Python backend developers with little support.

VibeStack is our answer to this. It is a harness that turns a plain-English description of an application into a working, tested, full-stack web application with a Python backend (FastAPI, Flask or Django) and a React frontend. Instead of one large prompt, the work is split across eight agents wired together as a LangGraph state machine: an Architect, a Planner, a Code Generator, a Validator, an Error Classifier, a Reflector, a Packager and a Circuit Breaker. Three design decisions keep the output reliable. First, everything that can be written once and verified (models, schemas, routers, authentication and the frontend) comes from tested templates, and the language model writes only the business logic. Second, every generated application is executed in an isolated sandbox and must pass four gates in order: static analysis, import, boot and tests. The tests include contract tests, edge-case tests derived from the design without any AI, and business-rule tests that are written before the code exists. Third, when a gate fails, a self-healing loop classifies the error, locates the faulty file and repairs it with a tool-using agent. The loop does not stop after a fixed number of tries. It continues while each repair makes measurable progress, escalates from a cheap model to a strong model to the verified template when it stalls, and rolls back repairs that make things worse. Every agent action is recorded in a change ledger with the exact diff.

At the time of writing, the harness, the web platform and an automated test suite of about 120 tests are complete and merged. In the interim evaluation, a real end-to-end run produced a working blog backend with JWT authentication in about 51 seconds for less than one US cent, and all ten benchmark projects in the harness evaluation were repaired successfully. A full benchmark of fifteen prompts on real models, with ablation studies, is the main task for the next phase.

Keywords: Large language models, multi-agent systems, code generation, self-healing software, automated testing, LangGraph, FastAPI.

Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

List of Figures viii

List of Tables ix

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 3
- 1.5 Relevance to Sustainable Development Goals . . . 3
- 1.6 Organisation of the Report . . . 4

2 Literature Review 5
- 2.1 Introduction . . . 5
- 2.2 Related Work . . . 5
  - 2.2.1 Existing System . . . 5
  - 2.2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (N. Shinn et al., NeurIPS, 2023) . . . 6
  - 2.2.3 Teaching Large Language Models to Self-Debug (X. Chen, M. Lin, N. Schärli and D. Zhou, ICLR, 2024) . . . 6
  - 2.2.4 CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing (Z. Gou et al., ICLR, 2024) . . . 7
  - 2.2.5 ReAct: Synergizing Reasoning and Acting in Language Models (S. Yao et al., ICLR, 2023) . . . 7
  - 2.2.6 SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (C. E. Jimenez et al., ICLR, 2024) . . . 7
  - 2.2.7 SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering (J. Yang et al., NeurIPS, 2024) . . . 8
  - 2.2.8 AgentCoder: Multi-Agent Code Generation with Iterative Testing and Optimisation (D. Huang et al., arXiv:2312.13010, 2023) . . . 8

2.2.9 Evaluating Large Language Models Trained on Code (M. Chen et al., arXiv:2107.03374, 2021) . . . 9
2.2.10 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (S. Hong et al., ICLR, 2024) . . . 9
2.2.11 ChatDev: Communicative Agents for Software Development (C. Qian et al., ACL, 2024) . . . 9
2.3 Gaps in Literature . . . 10

3 System Design **12**
3.1 System Architecture . . . 12
3.2 Modules . . . 13
3.2.1 Architect (Requirement Analysis and Design) . . . 15
3.2.2 Planner and Design Review . . . 16
3.2.3 Code Generator . . . 16
3.2.4 Validator (Sandbox Execution) . . . 17
3.2.5 Error Classifier . . . 17
3.2.6 Fault Localisation . . . 18
3.2.7 Reflector (Tool-Using Repair Agent) . . . 19
3.2.8 Healing Controller (When to Stop) . . . 19
3.2.9 Packager and Circuit Breaker . . . 21
3.2.10 Supporting Mechanisms . . . 21
3.3 System Diagrams . . . 22
3.3.1 Use Case Diagram . . . 22
3.3.2 Sequence Diagram . . . 23
3.3.3 Data Flow Diagram . . . 23
3.3.4 Entity-Relationship Diagram . . . 25
3.4 Hardware and Software Specifications . . . 25

4 System Implementation **27**
4.1 Technology Stack . . . 27
4.2 Repository Organisation . . . 27
4.3 Agent Orchestration . . . 29
4.4 Self-Healing Implementation . . . 29
4.5 Generated Applications . . . 30
4.6 The Platform . . . 31
4.7 User Interface . . . 32
4.8 Testing Approach . . . 33

5 Preliminary Results **35**
5.1 Automated Test Suite . . . 35
5.2 Benchmark Suite . . . 35
5.3 Interim Measurements (Review 1) . . . 36
5.4 Performance Indicators . . . 37
5.5 Planned Evaluation and Ablation Design . . . 37

6 Project Schedule **39**

7 Conclusion **41**
7.1 Limitations 41
7.2 Future Work 42

A Turnitin Plagiarism Report **45**

B Base Paper (most relevant one) **46**

List of Figures

3.1 Overall system architecture of VibeStack 12
3.2 Flowchart of the Generate → Validate → Heal algorithm 14
3.3 Module block diagram 14
3.4 UML use case diagram 23
3.5 UML sequence diagram of a build 24
3.6 Data flow diagram 24
3.7 Entity-relationship diagram of the platform database 25

4.1 Home page with the guided prompt builder (offline demo mode) 32
4.2 Design review before code generation (offline demo mode) 33
4.3 Project workspace with the agent pipeline and the live app preview (offline demo mode) 33
4.4 Live view during a build: agent progress, activity log and the code being written (offline demo mode) 34

6.1 Gantt chart of the project 39

List of Tables

2.1 Comparison of existing systems with the proposed system 11

3.1 Error classes used by the Error Classifier, in matching order 18
3.2 Summary of modules and the techniques they use 22
3.3 Hardware specifications 25
3.4 Software specifications 26

4.1 Technology stack and reasons for selection 28

5.1 Automated tests by area (main branch) 36
5.2 Interim measurements reported at Review 1 37
5.3 Expected and achieved performance indicators 37

6.1 Milestones completed 40
6.2 Remaining work and planned timeline 40

Chapter 1

Introduction

1.1 Background

Large language models (LLMs) trained on source code have changed how software is written. Tools such as GitHub Copilot [1] complete functions as the developer types, and newer "app builders" promise to produce an entire application from a short description. Research systems have pushed the same idea further. Codex [2] showed that a model can solve a large share of short programming problems, and multi-agent systems such as MetaGPT [3] and ChatDev [4] split software development into roles played by different agents.

In our own use of these tools during earlier coursework and hackathons, we kept running into the same pattern. The first draft looks complete, but it has not been run. When it is run, something small breaks: an import points to a file that was never created, a database relation is defined on one side only, or an endpoint returns the wrong status code. Each of these is easy to fix once found, but finding them in a project of twenty or thirty files written by someone else (in this case, a model) is slow. The time saved by generating the code is often lost again while debugging it.

A second observation shaped the project. The best-known app builders target JavaScript stacks such as React with Node.js. Python is the most common language for data and AI work, and FastAPI, Flask and Django are widely used for backends, yet

there was no comparable tool that understood these frameworks and their conventions natively and that a developer could run on their own machine or server.

VibeStack started from these two observations. Rather than trying to make the model write better code in one attempt, we built a harness around it that does what a careful developer would do: plan the design, write the code, run it, read the error, fix the right file, and run it again. The model is only one part of the system. The rest is ordinary, deterministic software that checks the model's work.

1.2 Problem Statement

To address the lack of AI-powered, deployment-ready application builders tailored for Python backend developers by overcoming the JavaScript-centric bias of existing tools, and to build a reliable, locally installable or deployable backend generation system that natively understands Python frameworks and conventions, giving developers full control over their infrastructure and code without dependency on third-party cloud services.

Stated as an engineering problem: AI code generators produce unverified and often inconsistent multi-file backends. We need a harness that designs, executes, tests and repairs generated code *before* it is delivered, and that explains every change it makes.

1.3 Objectives

The objectives of the project are:

1. To design a multi-agent pipeline of eight specialised agents that converts a natural-language application description into a deployable full-stack web application, with a React frontend and a FastAPI, Flask or Django backend.

2. To verify generated applications automatically through static analysis, sandboxed execution, design-derived contract and edge-case tests, business-rule tests written before the code, and a frontend–backend contract check.

3. To repair defects autonomously with a progress-driven self-healing loop that needs no fixed iteration limit, using a tool-augmented repair agent that can inspect code and run tests.

4. To support iterative development, in which follow-up requests produce new, versioned applications that reuse unchanged components.

5. To evaluate the system on a benchmark of 15 applications across three frameworks, with ablations that measure the contribution of dynamic healing, tool use and design-derived tests to success rate, escaped defects, cost and time.

6. To make the system extensible and interoperable through a plugin interface and the Model Context Protocol (MCP), as both server and client.

1.4 Scope

The project focuses on CRUD-style web applications with business rules: systems built around entities, relations, permissions and actions such as booking, checkout or approval. Generated applications use SQLAlchemy 2.0 (FastAPI and Flask) or the Django ORM, JWT-based authentication [5], and a React frontend. Each one comes with tests, database migrations, a Dockerfile and a CI configuration. The platform itself is a web application with live progress, a live preview of the generated app and one-click deployment. Mobile apps, desktop software and arbitrary algorithmic programs are outside the scope.

1.5 Relevance to Sustainable Development Goals

Following the suggestion of the zeroth review committee, the project is mapped to a single goal: **SDG 9, Industry, Innovation and Infrastructure** [6], specifically **Target 9.5**, which calls for enhancing research and upgrading the technological capabilities of industry. VibeStack makes reliable, verified software engineering accessible to students, small teams and start-ups that cannot afford a large engineering team.

1.6 Organisation of the Report

Chapter 2 reviews existing tools and research on LLM-based code generation and self-repair and identifies the gap this project addresses. Chapter 3 presents the system architecture, the modules and the algorithms used in each. Chapter 4 describes how the system has been implemented. Chapter 5 reports the preliminary results. Chapter 6 gives the project schedule, and Chapter 7 concludes the report and outlines the work planned for Phase II.

Chapter 2

Literature Review

2.1 Introduction

Our literature survey covered fifteen sources: research papers on LLM agents, self-correction and automated software engineering, plus the documentation of the frameworks we build on. This chapter summarises the most relevant ones. For each paper, we note the core idea, how it was evaluated and the limitation that matters for our problem. We close the chapter with the gaps we found and a comparison against our proposed system.

2.2 Related Work

2.2.1 Existing System

Existing tools fall roughly into three groups.

Code completion assistants such as GitHub Copilot [1] work inside the editor and suggest the next few lines or a whole function. They are very good at local code but have no view of the whole project and never run what they write. Checking the result is left entirely to the developer.

Prompt-to-project generators such as gpt-engineer [7] take a description and write a full set of files. Some ask clarifying questions first. In most of them, however, the output

is generated in one pass and handed over without being executed, so the developer receives a project that may not even start.

Hosted app builders generate a running application inside the vendor's cloud and are mostly built around JavaScript stacks. They do run the code, but the developer depends on the vendor's infrastructure, has limited control over the backend, and gets little explanation of why the code looks the way it does.

None of these groups combines a Python-first backend, execution-based verification, automatic repair and a record of what was changed and why.

2.2.2 Reflexion: Language Agents with Verbal Reinforcement Learning (N. Shinn et al., NeurIPS, 2023)

Reflexion [8] lets an agent learn from failure without updating model weights. After each attempt, the agent receives feedback (for code, the result of unit tests), writes a short verbal reflection on what went wrong, and stores it in memory for the next attempt. On the HumanEval coding benchmark this raised the pass rate well above a single attempt.

Limitations: Reflexion works on single functions with given tests. The number of trials is fixed in advance, and the agent edits the whole solution rather than locating the faulty part. It does not address multi-file projects in which the error and its cause are in different files.

2.2.3 Teaching Large Language Models to Self-Debug (X. Chen, M. Lin, N. Schärli and D. Zhou, ICLR, 2024)

Self-Debug [9] teaches a model to debug its own code through few-shot prompting. The model executes the code, explains it line by line ("rubber duck debugging") and uses the execution result to produce a corrected version. The paper reports consistent gains on text-to-SQL, code translation and Python programming tasks.

Limitations: The technique is evaluated on self-contained snippets. There is no notion of a project layout, of which file to fix, or of when to stop when repairs no longer

help.

2.2.4 CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing (Z. Gou et al., ICLR, 2024)

CRITIC [10] argues that models are poor at judging their own output from introspection alone, but improve when they can verify it with external tools such as a search engine or a code interpreter. The model generates an answer, uses tools to critique it, and revises it.

Limitations: CRITIC shows why tools matter for correction, but it is a general framework evaluated on question answering, mathematics and toxicity. It does not deal with building and testing software projects.

2.2.5 ReAct: Synergizing Reasoning and Acting in Language Models (S. Yao et al., ICLR, 2023)

ReAct [11] interleaves reasoning steps with actions such as calling a tool and observing the result. This pattern is now the basis of most tool-using agents, including our repair agent, which reads files, searches code and runs tests before proposing a fix.

Limitations: ReAct is a prompting pattern, not a software engineering system. It gives no guidance on verification, test oracles or limiting the cost of long loops.

2.2.6 SWE-bench: Can Language Models Resolve Real-World GitHub Issues? (C. E. Jimenez et al., ICLR, 2024)

SWE-bench [12] is a benchmark of real GitHub issues from popular Python repositories. A model must produce a patch that makes the repository's hidden tests pass. It showed that resolving real issues in large codebases is much harder than solving isolated problems.

Limitations: SWE-bench evaluates the repair of existing human-written code. It does not cover generating a new project, and it assumes a trustworthy test suite already

exists. In generation from scratch, the tests themselves must also be produced and checked.

2.2.7 SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering (J. Yang et al., NeurIPS, 2024)

SWE-agent [13] shows that the interface given to an agent matters as much as the model. With a small set of well-designed commands for viewing, searching and editing files, plus feedback from running code, a model resolves many more SWE-bench issues than with a raw shell.

Limitations: SWE-agent edits existing repositories and stops based on a cost or step budget. Our Reflector borrows its idea of a small, purpose-built toolset, but applies it to code that the system generated itself.

2.2.8 AgentCoder: Multi-Agent Code Generation with Iterative Testing and Optimisation (D. Huang et al., arXiv:2312.13010, 2023)

AgentCoder [14] uses three agents: a programmer, a test designer and a test executor. The test designer writes tests independently of the programmer, so the tests are not biased by the code, and the executor feeds failures back to the programmer.

Limitations: The tests are still written by a language model, so they can be wrong, and nothing checks them. Evaluation is on function-level benchmarks rather than multi-file applications, and the loop runs for a fixed number of iterations.

2.2.9 Evaluating Large Language Models Trained on Code (M. Chen et al., arXiv:2107.03374, 2021)

This paper introduced Codex and the HumanEval benchmark, and established functional correctness, measured by running unit tests (pass@k), as the standard way to evaluate generated code instead of text similarity.

Limitations: HumanEval problems are single functions. The paper itself notes that models struggle with long specifications and multi-step logic, which is exactly where application generation lives.

2.2.10 MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework (S. Hong et al., ICLR, 2024)

MetaGPT [3] encodes standard operating procedures from human software companies into a team of agents (product manager, architect, engineer, QA) that exchange structured documents rather than free chat. Structured hand-offs reduced the inconsistencies seen in chat-based multi-agent systems.

Limitations: The generated projects are not always executable, and the QA step relies on the model rather than on independent checks. There is no explicit repair strategy with a stopping rule.

2.2.11 ChatDev: Communicative Agents for Software Development (C. Qian et al., ACL, 2024)

ChatDev [4] models a virtual software company in which agents in different roles talk through design, coding and testing phases. It produces small applications cheaply and quickly.

Limitations: The output is mostly small single-user programs. Verification is conversational, and the cost of long dialogues grows quickly. There is no target framework or deployment story for backend services.

2.3 Gaps in Literature

Reading these works together, we found five gaps that matter for building reliable backends:

1. **Scale of the target.** Most self-repair research works on single functions. Multi-file applications, where the error message and its cause are in different files, are rarely addressed.

2. **When to stop.** Almost every repair loop uses a fixed number of attempts or a step budget. A fixed limit either gives up on a repair that was making progress or keeps paying for one that is stuck.

3. **Trustworthy tests.** When the model writes both the code and the tests, a wrong test can drive the repair in the wrong direction. Few systems have an independent oracle that does not depend on the model.

4. **How much the model writes.** Existing generators ask the model to write everything, including boilerplate that could be written once and verified.

5. **Explainability and ownership.** Generated code is usually handed over without a record of why it was written or changed, and hosted tools keep the code and infrastructure in the vendor's cloud.

Table 2.1 compares representative systems with VibeStack against these points.

VibeStack addresses these gaps by (i) generating complete multi-file applications but letting the model write only the business logic, (ii) replacing the fixed attempt limit with a progress-based stopping rule, (iii) deriving edge-case tests from the design without any AI, so they act as an independent oracle, (iv) calibrating the model-written tests against verified code before use, and (v) recording every change in a ledger, with all code staying under the developer's control.

Table 2.1: Comparison of existing systems with the proposed system

System	**Multi-file app**	**Runs the code**	**Self-repair**	**Stopping rule**	**Independent test oracle**	**Change ledger**
Copilot [1]	No	No	No	–	No	No
Reflexion [8]	No	Yes	Yes	Fixed trials	No	No
Self-Debug [9]	No	Yes	Yes	Fixed turns	No	No
AgentCoder [14]	No	Yes	Yes	Fixed iterations	No (LLM tests)	No
MetaGPT [3]	Yes	Partly	Limited	–	No	Documents only
SWE-agent [13]	Existing repo	Yes	Yes	Cost budget	Given tests	Trajectory log
VibeStack	**Yes, full-stack**	**Yes, sand-boxed**	**Yes, tool-using**	**Progress-driven**	**Yes, design-derived**	**Yes, with diffs**

Chapter 3

System Design

3.1 System Architecture

VibeStack is organised into four layers, shown in Fig. 3.1.

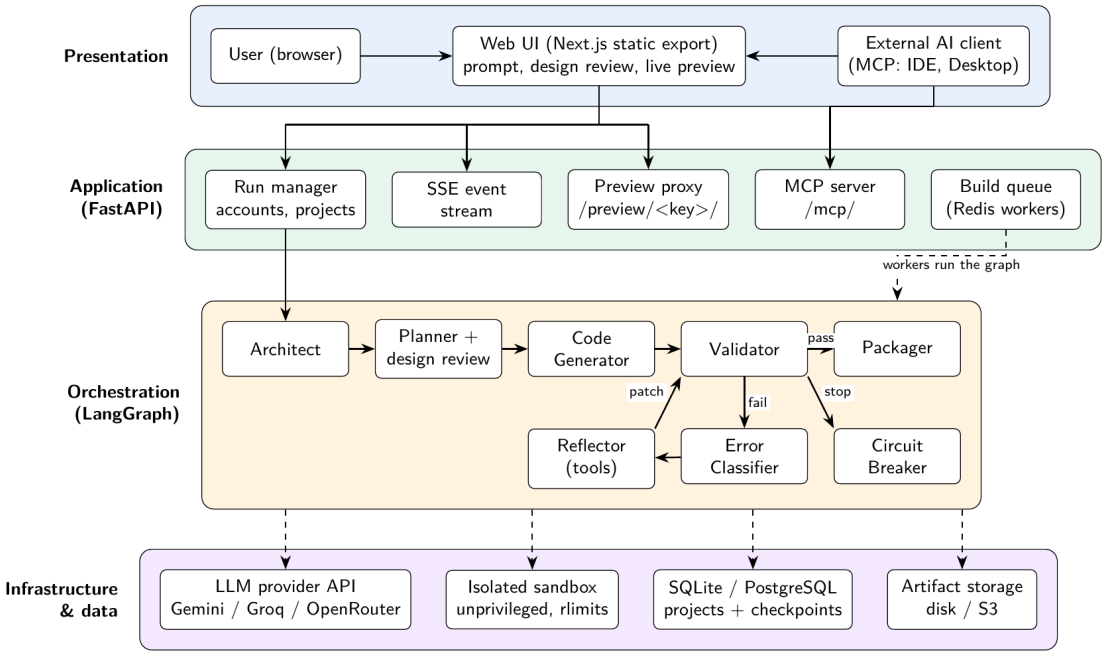


Figure 3.1: Overall system architecture of VibeStack

1. **Presentation layer.** A Next.js web interface [15] where the user describes the application, reviews the design, watches the agents work, inspects the code, the tests and the ledger, and uses the generated app in a live preview. The interface

is exported as static files and served by the backend, so a single container runs the whole platform.

2. **Application layer.** A FastAPI service [16] that manages accounts, projects and versions. It starts builds through the run manager, streams agent progress to the browser with Server-Sent Events (SSE), and proxies the live preview of each generated app at a private `/preview/<key>/` address. It also exposes an MCP server so that external AI tools can drive VibeStack [17].

3. **Orchestration layer.** The eight agents, connected as a LangGraph [18] state machine. LangGraph was chosen because the self-healing cycle is a loop with conditional exits, and because it saves a checkpoint after every step, so an interrupted build can resume where it stopped.

4. **Infrastructure and data layer.** The LLM provider API (Gemini by default, or Groq, OpenRouter [19] or any OpenAI-compatible endpoint), the isolated sandbox that executes generated code, the checkpoint and platform database (SQLite locally, PostgreSQL in production) and the storage for generated artifacts (local disk or any S3-compatible bucket).

The flow through the orchestration layer is shown in Fig. 3.2. A build moves from the Architect to the Planner, the Code Generator and the Validator. If every gate passes, the Packager delivers the app. If a gate fails and healing is still making progress, the Error Classifier and the Reflector repair the code and send it back to the Validator. If healing stops making progress, the Circuit Breaker ends the run with a diagnostic report.

3.2 Modules

Fig. 3.3 shows the main modules. Each agent is a single Python function in `backend/agent/nodes.py` that reads the shared state and returns the fields it changes. This section describes what each module does and the algorithm behind it. We give the file and function names so that every step can be checked against the source code.

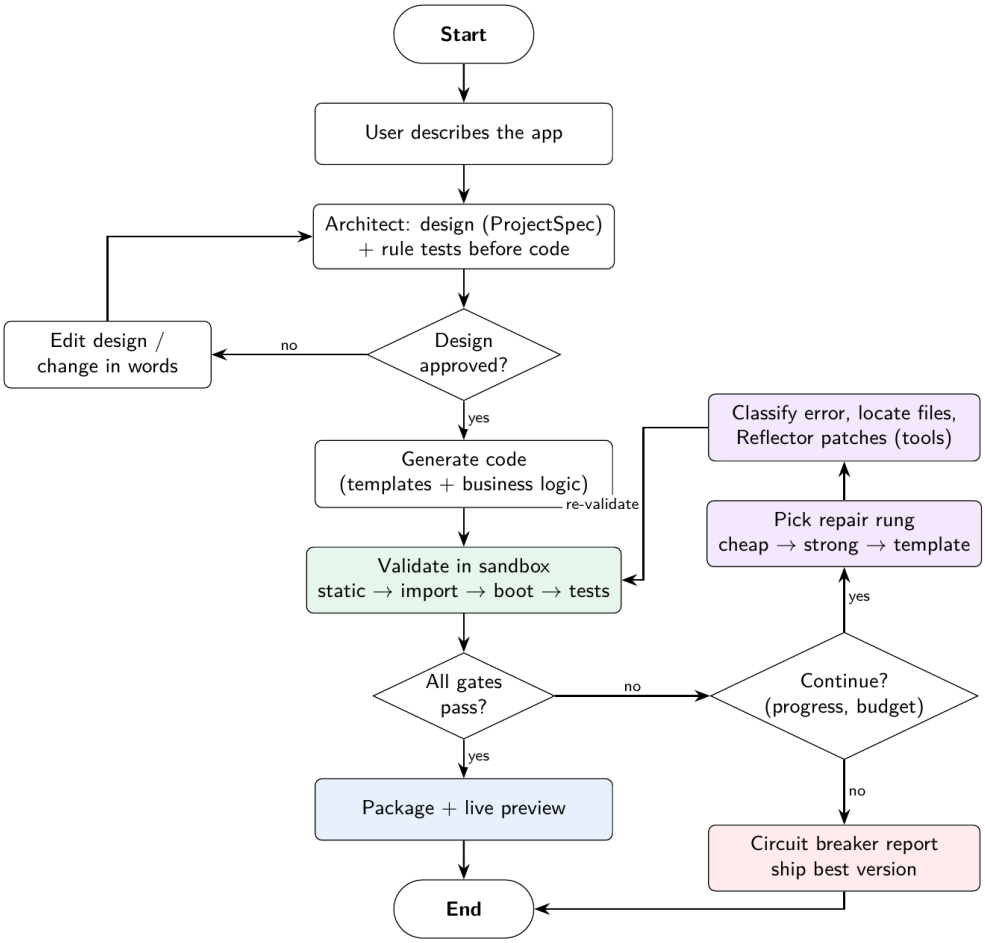


Figure 3.2: Flowchart of the Generate → Validate → Heal algorithm

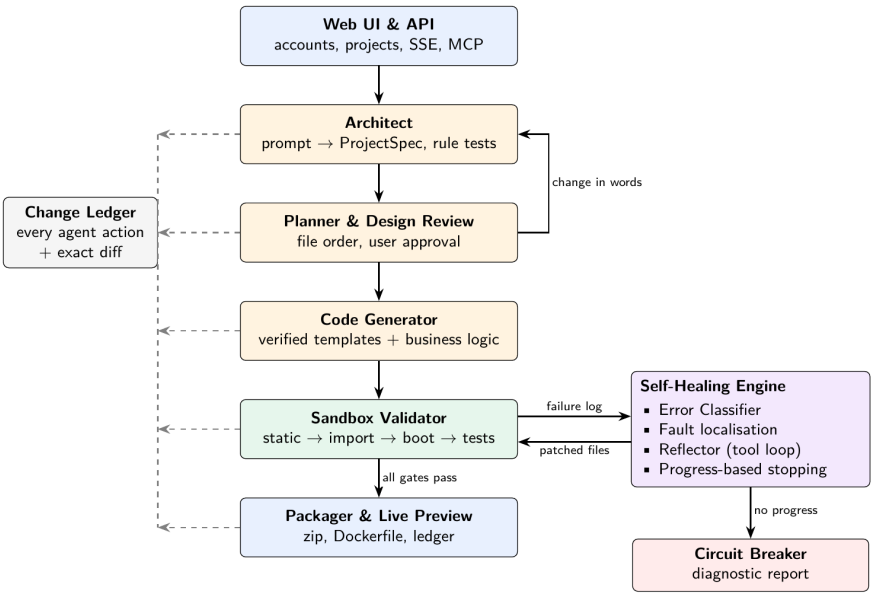


Figure 3.3: Module block diagram

3.2.1 Architect (Requirement Analysis and Design)

The Architect (`parse_spec`) converts the user's description into a structured design called a `ProjectSpec` (`agent/state.py`). The spec is a Pydantic [20] model, so any output that does not match the schema is rejected immediately rather than discovered later as broken code. A spec contains:

- **Entities** with typed fields (`str`, `text`, `int`, `float`, `decimal`, `bool`, `date`, `datetime`, `json`), uniqueness, indexes and foreign keys with delete rules (`cascade`, `restrict`, `set_null`).
- **State machines** on enum fields, for example `draft` → `running` → `done`. A forbidden transition must return HTTP 409.
- **Access rules** per entity (`any_user`, `owner` or `admin`) and named **roles**.
- **Custom actions** with business rules, such as `POST /orders/{id}/checkout`.
- A numbered list of **requirements**, each linked to the entity or action that covers it, and the **plugins** the build needs, with a reason for each.

Algorithm 1: Architect

1. Send the prompt, any project rules and, for a follow-up, the previous design to the strong model with the design prompt.
2. Parse the JSON reply into a `ProjectSpec`. If parsing fails, send the validation error back once for a repair round-trip.
3. Normalise the spec, for example by turning on authentication when an entity needs owners or admins.
4. If the spec has custom actions, ask the model to write **business-rule tests** now, before any code exists (`testwriter.write_rule_tests`).
5. For a follow-up request, compute what changed between the old and new design (`design_changes`) so that only the changed parts are regenerated.

3.2.2 Planner and Design Review

The Planner (`plan`) orders the files to be generated by dependency: database and configuration first, then models, schemas, services and routers, then the application entry point and the tests. When design review is turned on, the build pauses here. The user sees an entity diagram, a role-by-table permission matrix, the feature toggles and the actions, and can approve the design, edit it, or ask for a change in words. A change in words sends the build back to the Architect. No code is generated until the design is approved.

3.2.3 Code Generator

The most important design decision in VibeStack is how little code the model writes. Models, schemas, routers, authentication, error handling and the React frontend follow patterns that do not change from one app to the next, so they are produced from verified Jinja2 templates (`agent/templates/`). These templates have been tested against the contract tests for all three frameworks. The model writes only the **business logic**: one service file per entity that has custom actions (`app/services/<entity>.py`), plus a small `ui.json` that controls the look of the app.

Algorithm 2: Code Generator

1. Render every template file for the chosen framework (`codegen.py`, `stacks.py`).
2. Derive the **edge-case tests** from the design, without any model: an unknown foreign key must return 409, a forbidden state change 409, another user's row 404, an out-of-range value 422, and so on.
3. For each entity with actions, ask the model, in parallel, to write the service file, giving it only that entity's context (`_generate_service`).
4. Check each generated file with Python's `ast` parser. A syntax error gets one cheap repair call before validation (`_fix_syntax`).
5. Store the template output separately, so the healing loop can later restore any file to its verified version.

Because only the business-logic files are model-written, the helper `healing.editable()` lets the repair agent change only those files and `ui.json`. Everything else is protected.

3.2.4 Validator (Sandbox Execution)

The Validator (`validate`, `agent/sandbox.py`) writes the project to a temporary directory and runs four gates, cheapest first. The first gate that fails stops validation, and its log is passed on.

1. **Static gate.** AST checks from `agent/bughunt.py` for `eval`/`exec`, shell commands, hard-coded secrets, SQL built from strings, hard-coded debug mode, bare `except` and stray `print`. The frontend contract check (`frontend.check_frontend`) runs here too: everything the React app uses must exist in the backend.

2. **Import gate.** Import the application module. This catches syntax errors, missing modules and broken ORM mappings.

3. **Boot gate.** Start the server on a free port and wait for `/health` to answer.

4. **Test gate.** Run pytest [21] with the smoke and contract tests, the design-derived edge-case tests and the calibrated rule tests. Database migrations are generated and applied once, to prove that they work.

Generated code is untrusted, so the sandbox runs it as a separate unprivileged user with a scrubbed environment (no API keys) and operating-system limits set through `setrlimit`: CPU time, 2 GB of address space by default, 100 MB per file, 512 open files and a cap on processes. Each gate has a timeout (60 s by default), and the test gate gets five extra seconds per table.

3.2.5 Error Classifier

The Error Classifier (`classify`) does not use a language model. It is a list of regular expressions in `healing.py`, checked in order, where the first match wins (Table 3.1). This makes it fast, free and deterministic, and every rule is covered by unit tests. The class

decides what context the Reflector receives. It also separates real bugs from *transient* failures, such as the sandbox running out of threads, which are retried without counting against the code.

Table 3.1: Error classes used by the Error Classifier, in matching order

Class	**Typical evidence in the log**
sandbox	"can't start new thread", too many open files (transient, not a bug)
frontend	`FrontendContractError` from the contract check
static	`StaticCheckError` from the AST checks
timeout	a gate exceeded its time limit
syntax	`SyntaxError`, `IndentationError`
dependency	`ModuleNotFoundError` for a third-party package
import	`ImportError`, "cannot import name"
orm	`sqlalchemy.exc`, missing foreign keys, mapper errors
validation	Pydantic validation errors, unexpected HTTP 422
runtime	`NameError`, `AttributeError`, `TypeError`, `KeyError`
contract	failed assertions in the tests
unknown	none of the above

3.2.6 Fault Localisation

A traceback often ends inside template code even though the real mistake is in the business logic that called it. Localisation (`locate_failing_file`) therefore uses several clues in a fixed order:

1. a file named by the frontend contract check;
2. the deepest stack frame that lies inside the generated `app/` folder;
3. for a failing rule or edge-case test, the entity and action whose URL the test calls (`file_for_test`);
4. for a failing CRUD smoke test, that entity's router;

5. any generated file mentioned in the log.

The result is then redirected from protected template code to the entity's service file (`repair_target`). Because complex apps often fail in several independent places, `failing_targets` collects up to three distinct files with failing tests, and all of them are repaired in the same attempt.

3.2.7 Reflector (Tool-Using Repair Agent)

The Reflector (`reflect`) repairs the located files. Following the ReAct and SWE-agent line of work [11, 13], it does not answer in one shot. It has a small toolbox (`agent/tools.py`): `list_files`, `read_file`, `search_code`, `list_routes`, `write_file` (which stages a change), `run_tests` and `check_frontend`. It can make at most six tool calls per repair, through the provider's function-calling interface. In practice it reads the failing test, stages a fix, runs the failing tests against the staged fix and only then answers. Tools from external MCP servers and plugins can be added to the toolbox. Every call is written to the ledger.

The context given to the model is kept small. It contains the failing file, its sibling files for the same entity (`related_files`), the source of the failing tests and the tail of the log. This keeps cost low and stops the model from "fixing" unrelated code.

If the Reflector believes that a model-written rule test contradicts the requirements, it can dispute it. A single strong-model call acting as a judge (`testwriter.judge`) decides, and the decision is logged. Contract and edge-case tests can never be removed.

3.2.8 Healing Controller (When to Stop)

Instead of the usual fixed limit of three attempts, VibeStack keeps repairing while repairs make progress. After each validation, a small record is kept: the gate reached, the number of failing tests, the names of the failing tests, a normalised error signature and a SHA-256 fingerprint of all files (`history_entry`). A result is scored as a pair (gate reached,

−failing tests), where a later gate is better and fewer failures is better:

$$score(v) = \Big(\text{index}(gate_v),\ -failing_v\Big), \quad gate \in \{\text{static}, \text{import}, \text{boot}, \text{tests}\}.$$

Algorithm 3: Progress test (`made_progress`)

A new validation v counts as progress if any of the following holds:

1. its score is the best so far, or better than the previous attempt's;
2. it fails as many tests as before, but at least one previously failing test now passes;
3. its error signature has never been seen before (the old error was fixed and a new one surfaced).

A validation that scores worse than the previous one is never progress, and that repair is rolled back to the best version (`regressed`).

Algorithm 4: Repair ladder and stopping rule (`repair_rung`, `should_continue`)

1. Count the consecutive attempts without progress (s).
2. Choose the repair method from the ladder {cheap model, strong model, verified template} using s as the index. Any progress resets s to 0, so a new problem starts again with the cheap model.
3. Stop when any of these holds:
   - all three rungs have been tried without progress;
   - the file fingerprint repeats, meaning the repairs are going in circles;
   - the money or token budget is used up;
   - more than three transient failures happen in a row;
   - a safety limit on attempts is reached. This is a seatbelt, not the rule: it is 12 by default and grows with the number of service files, up to 30.
4. On stopping, ship the **best** version, not the last one.

3.2.9 Packager and Circuit Breaker

When all gates pass, the Packager (`package`) bundles the project as a zip. The zip contains the source, tests, migrations, a non-root Dockerfile with a health check, a docker-compose file with PostgreSQL, a Makefile, a GitHub Actions workflow, a README and `VIBESTACK_LEDGER.md`. The Packager then starts the app as a live preview. When healing stops without success, the Circuit Breaker (`failure_report`) writes a diagnostic report instead: the last failing gate, the error class, what was tried and why healing stopped. From there the user can choose **Keep fixing**, which continues healing the same code, or **Rebuild**.

3.2.10 Supporting Mechanisms

Rule-test calibration. The model writes business-rule tests before the code exists, and some of them may be wrong. Before they are used, they are run once against the verified template code (`testwriter.calibrate`). A test about plain CRUD behaviour that the verified templates fail contradicts the contract, so it is removed before it can mislead the repairs.

Frontend contract check. The generated React app is driven by `config.json`, derived from the design, and `ui.json`, written by the model. The Validator checks that every entity, field and action the frontend refers to exists in the backend. A mismatch is healed like any other bug.

Change ledger. Every agent appends entries to the ledger (`agent/ledger.py`) recording which agent acted, on which file, why, which model and tools it used, and the exact diff. The ledger is shown in the interface and shipped inside the zip.

LLM client. Agents ask for a *role* ("strong" or "cheap") rather than a specific model (`agent/llm.py`). If the routed model fails, the call is retried once on a fallback model. Rate-limit errors (HTTP 429) are retried with back-off, every call has a deadline, and token use and cost are tracked per run.

Table 3.2 summarises the modules.

Table 3.2: Summary of modules and the techniques they use

Module	**Source**	**Uses LLM?**	**Technique**
Architect	`nodes.parse_spec`	Yes (strong)	Schema-validated structured output, rule tests before code
Planner	`nodes.plan`	No	Dependency ordering, human design review
Code Generator	`nodes.generate`, `codegen.py`	Yes, services only	Verified templates, parallel generation, AI-free edge tests
Validator	`nodes.validate`, `sandbox.py`	No	AST checks, sandboxed import/boot/test gates
Error Classifier	`healing.classify_error`	No	Ordered regular-expression taxonomy
Reflector	`nodes.reflect`, `tools.py`	Yes (tiered)	Fault localisation, tool loop of at most 6 calls
Healing controller	`healing.should_continue`	No	Progress score, repair ladder, rollback
Packager	`nodes.package`	No	Zip, Dockerfile, ledger, live preview
Circuit Breaker	`nodes.failure_report`	No	Diagnostic report

3.3 System Diagrams

3.3.1 Use Case Diagram

Fig. 3.4 shows the main actors. The **developer** signs up, creates a project, describes the app, reviews and approves the design, watches the build, uses the live preview, sends follow-up requests, restores earlier versions, downloads the zip and deploys a passing version. The **administrator** manages accounts and quotas. An **external AI client**, such as an IDE assistant, can create and edit projects through the MCP server. The **LLM provider** and the **Kubernetes cluster** are external systems.

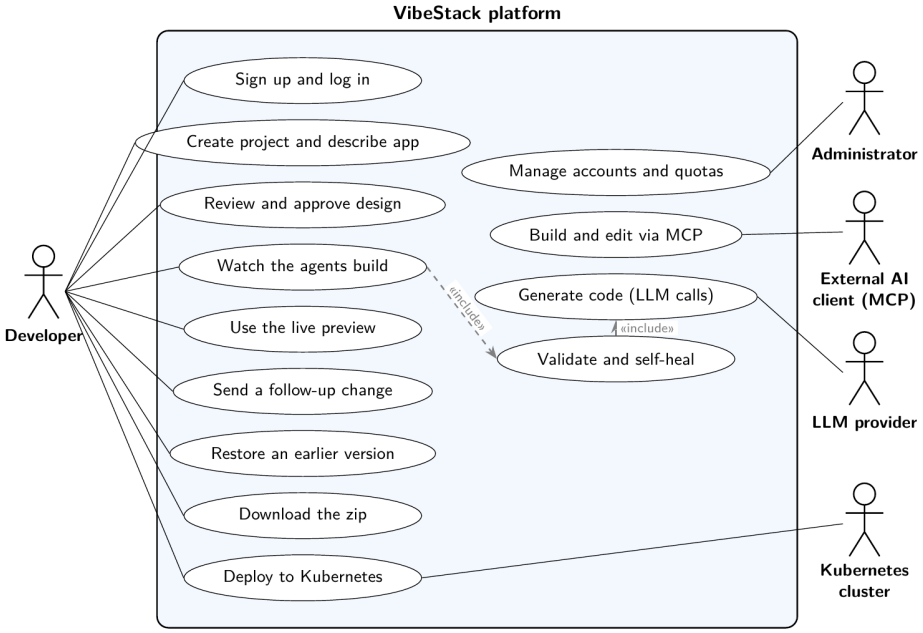


Figure 3.4: UML use case diagram

3.3.2 Sequence Diagram

Fig. 3.5 follows one build. The browser posts the prompt to the API, which creates a run and starts the LangGraph graph in a worker. Each agent's ledger entries are streamed back to the browser over SSE. The Validator calls the sandbox, and on failure the Classifier and Reflector loop runs, with the Reflector calling the LLM and its tools. When the run succeeds, the Packager stores the zip and registers the preview, and the browser loads the live app through the preview proxy.

3.3.3 Data Flow Diagram

At Level 0, the user's description goes in, and a verified application, a ledger and a live preview come out, with the LLM provider as an external entity. At Level 1 (Fig. 3.6), the data passes through the design (`ProjectSpec`), the file plan, the generated files, the validation results and the healing history, and is finally stored as an artifact and as a project version.

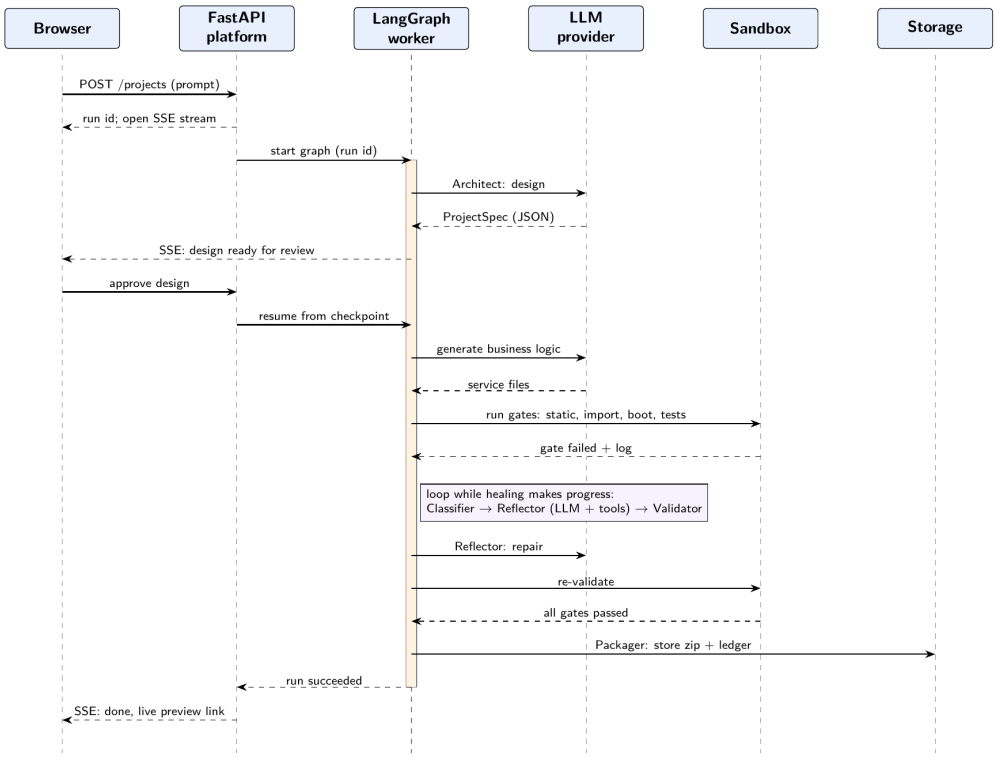


Figure 3.5: UML sequence diagram of a build

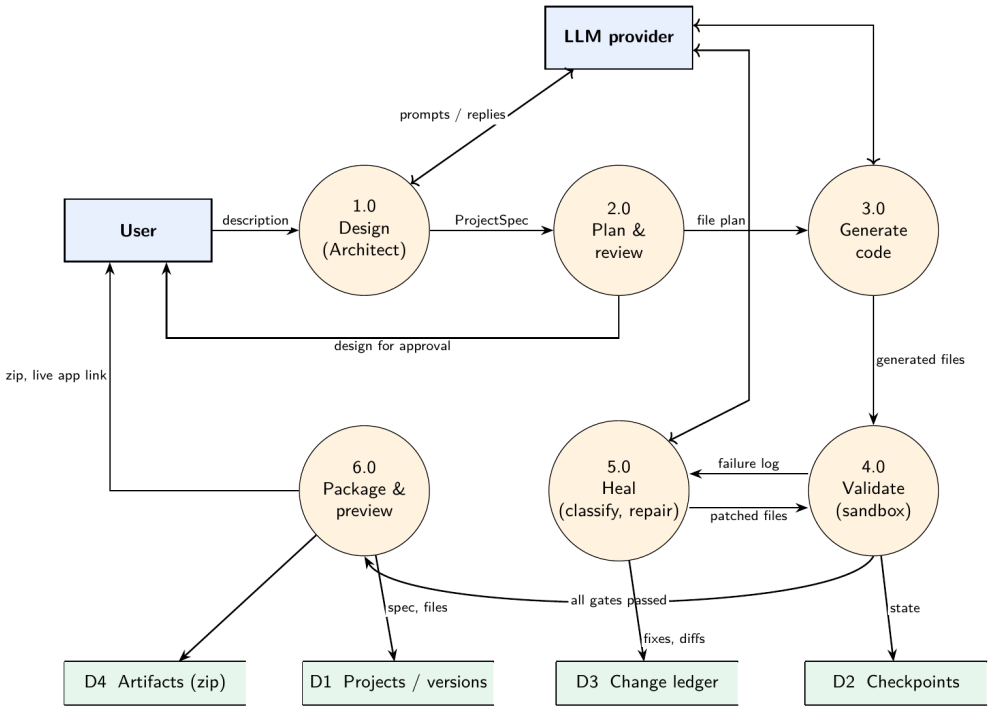


Figure 3.6: Data flow diagram

3.3.4 Entity-Relationship Diagram

The platform stores **users**, **projects** owned by users, **versions** (runs) belonging to a project, each with its spec, files, status, usage and artifact location, and the **ledger entries** of each run. LangGraph keeps its own **checkpoint** tables in the same database, keyed by run. Fig. 3.7 shows these relations.

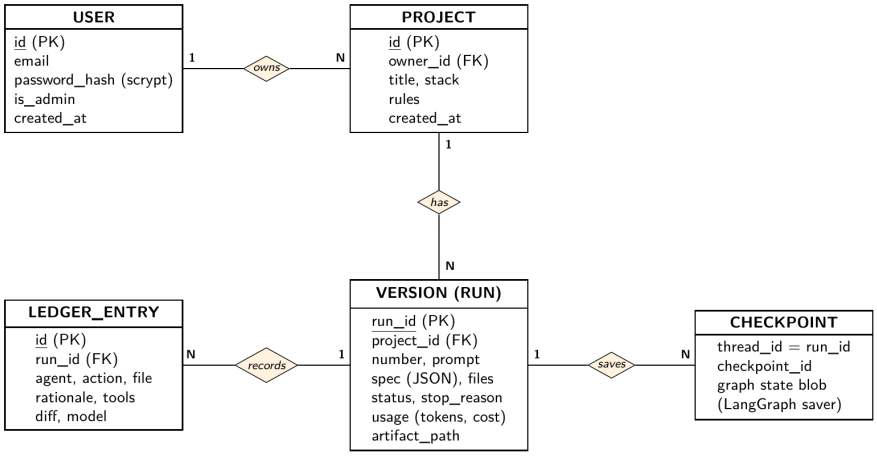


Figure 3.7: Entity-relationship diagram of the platform database

3.4 Hardware and Software Specifications

The language models are reached through hosted APIs, so no GPU is needed for development or deployment. Table 3.3 and Table 3.4 list the requirements.

Table 3.3: Hardware specifications

Item	**Specification**
Development machine	4-core CPU, 8 GB RAM, 10 GB free disk
Deployment (cloud)	Railway container, 1 vCPU, 1 GB RAM, persistent volume at `/data`
GPU	Not required (models are API-hosted)
Network	Internet access to the LLM provider API

Table 3.4: Software specifications

Item	**Specification**
Operating system	Linux (container), also runs on Windows/macOS for development
Languages	Python 3.11, TypeScript
Runtime	Node.js 22 (frontend build only)
Agent framework	LangGraph
Backend frameworks	FastAPI, SQLAlchemy 2.0, Pydantic v2; Flask and Django for generated apps
Frontend	Next.js, React, Tailwind CSS
Database	SQLite (local), PostgreSQL (production)
Testing	pytest, Playwright (browser smoke test)
Deployment	Docker, Docker Compose, Railway; Kubernetes manifests
LLM access	Gemini (default), Groq, OpenRouter or any OpenAI-compatible API

Chapter 4

System Implementation

This chapter describes how the design was turned into working software. All of the code described here is merged into the main branch of the project repository.

4.1 Technology Stack

Table 4.1 lists the main technologies and why each was chosen. One choice deserves a comment. The platform itself is written in FastAPI with SQLAlchemy and Pydantic, the same stack it generates. This meant that the patterns we wrote into the templates were patterns we were already using and testing every day.

4.2 Repository Organisation

The code is organised so that each idea lives in one file:

```
backend/
  agent/          the harness
    graph.py        LangGraph wiring of the 8 agents + CLI
    nodes.py        one function per agent
    state.py        the design (ProjectSpec) and graph state
    healing.py      error classes, fault localisation, stopping rule
    tools.py        the Reflector's tools
    sandbox.py      import -> boot -> tests with OS limits
    bughunt.py      static checks
    testwriter.py   rule tests, calibration, judge
```

Table 4.1: Technology stack and reasons for selection

Category	**Choice**	**Why it was selected**
Languages	Python 3.11, TypeScript	Python for the AI ecosystem and as the output language; TypeScript for a type-safe UI
Backend framework	FastAPI, SQLAlchemy 2.0, Pydantic v2	Async, automatic API docs, typed models; the same stack we generate
Agent orchestration	LangGraph	State machine with loops and checkpoints, which suits the heal cycle
LLM access	Gemini, Groq, OpenRouter, OpenAI-compatible	One client with role-based routing, fallback and cost tracking
Frontend	Next.js, React, Tailwind CSS	Static export served by FastAPI; fast and projector-friendly
Database	SQLite / PostgreSQL	Zero-setup locally, robust in production; also holds the checkpoints
Protocols	HTTP/REST, SSE, JWT, MCP	SSE streams agent progress live; JWT secures apps; MCP connects AI tools
Libraries	Jinja2, httpx, uvicorn, pytest	Templates, HTTP calls, app server, tests
Deployment	Docker, Railway, Kubernetes	Low cost, Git-based deploys; Docker for self-hosting
Tools	VS Code, Git/GitHub, ruff, GitHub Actions	Shared tooling, linting, CI

```
    codegen.py      template rendering, edge-case tests
    stacks.py       FastAPI / Flask / Django specifics
    frontend.py     frontend config and contract check
    llm.py          provider client: tiers, fallback, cost
    templates/      verified templates
  app/            the platform: accounts, projects, runs, SSE,
                  previews, MCP server, queue and worker, deploy
  plugins/        example plugins
  evals/          benchmark prompts and runner
  tests/          automated tests
frontend/         Next.js user interface
deploy/k8s/       Kubernetes manifests
```

4.3 Agent Orchestration

The graph is built in `agent/graph.py`. Each of the eight agents is added as a node, and the edges encode the flow described in Chapter 3. The only decision point in the loop is `route_after_validate`, which sends the state to the Packager, the Error Classifier or the Circuit Breaker:

```
g.add_edge("parse_spec", "plan")
g.add_conditional_edges("plan",
    lambda s: "parse_spec" if s.get("design_feedback")
              else "generate")
g.add_edge("generate", "validate")
g.add_conditional_edges("validate",
    nodes.route_after_validate,
    ["package", "classify", "failure_report"])
g.add_edge("classify", "reflect")
g.add_edge("reflect", "validate")
```

A checkpointer (SQLite locally, PostgreSQL in production) saves the state after every node. This is what makes "resume after redeploy" and "keep fixing" possible: a run can be continued from its last checkpoint, or restarted at the Validator with the previous version's code.

4.4 Self-Healing Implementation

The healing rules are pure functions over the validation log, so they cost no tokens and can be unit-tested directly. The error classifier, for example, is simply an ordered list of patterns:

```
_RULES = [
  ("sandbox",  SANDBOX_LIMIT),
  ("frontend", re.compile(r"FrontendContractError")),
  ("static",   re.compile(r"StaticCheckError")),
  ("timeout",  re.compile(r"TimeoutError")),
  ("syntax",   re.compile(r"\b(SyntaxError|IndentationError)\b")),
  ...
```

```
  ("contract", re.compile(r"AssertionError|\bFAILED\b")),
]
```

The stopping rule reads the healing history and returns both a decision and a human-readable reason. The reason is shown to the user and written to the ledger, so there is never a silent stop:

```
def should_continue(history, tokens_used, token_budget,
                    llm_available, safety_limit, budget_problem=None):
    attempts = len(history) - 1
    if attempts >= safety_limit:
        return False, "safety limit reached"
    if attempts > 0 and budget_problem:
        return False, budget_problem
    ...   # repeated fingerprint, ladder used up -> stop
```

4.5 Generated Applications

Every generated project has the same layout in all three frameworks, so the same tests, error classifier and healing loop work unchanged. A generated project includes:

- **Backend:** models, schemas, services, routers, authentication (scrypt password hashing, JWT access tokens, optional rotating refresh tokens with reuse detection), RFC 7807 error responses [22], request IDs with JSON logs, `/health` and `/ready` endpoints, security headers and a Prometheus `/metrics` endpoint.
- **Optional production features**, each as verified template code with its own tests: account lockout after failed logins, an immutable audit log, token-bucket rate limiting (shared through Redis when configured), row locks for actions that move money or stock, and live updates over SSE.
- **Frontend:** a complete React and Tailwind app with no build step: login, a dashboard, a page per entity, forms built from the field rules, state and action buttons, and live updates.

- **Database:** Alembic or Django migrations, generated and applied once during validation.
- **Tests:** contract, edge-case and rule tests.
- **Delivery:** a Dockerfile, docker-compose, a Makefile, a CI workflow, a README and the ledger.

4.6 The Platform

Accounts, projects and versions. Users sign up and own projects. A project works like a chat: the first message builds version 1, and a follow-up such as "add reviews with a 1–5 rating" makes the Architect edit the design. Only the changed business logic is regenerated, the whole app is re-validated as version 2, and any earlier version can be restored. Storage uses SQLAlchemy Core over SQLite or PostgreSQL.

Live progress and preview. Agent progress is streamed over SSE. A draft preview built from the verified screens opens within about a minute, while the business logic is still being written. The final app runs in the sandbox behind a private `/preview/<key>/` link and is shown in desktop and phone views.

Guided prompts and project rules. The home page offers six starter apps and a step-by-step prompt builder. Per-project rules entered by the user are passed to every agent.

Plugins and MCP. A plugin is a single Python file that can add files, static checks, prompt rules or tools (`agent/plugins.py`). Nine example plugins are included, among them deploy configurations, seed data, OpenAPI and Postman export, and a secrets scan. The Architect chooses which plugins a build needs. VibeStack is also an MCP server at `/mcp/`, so assistants in Claude Desktop, Cursor or VS Code can build and edit projects, and it is an MCP client, so the Reflector can use tools from other MCP servers bundled in the container.

Scaling and operations. With `RUN_QUEUE=redis`, web servers queue builds in Redis and any number of workers build them. A worker that dies hands its runs to

the others, which continue from the last checkpoint. The platform has readiness checks, back-pressure, login rate limits, JSON logs, a Prometheus endpoint and optional Sentry. A passing version can be deployed to a connected Kubernetes cluster in one click. It gets its own namespace, PostgreSQL, a migration job, two replicas and an Ingress, and the public URL is smoke-tested before the app is marked live.

4.7 User Interface

The interface follows a prompt-first design. The home page (Fig. 4.1) takes the description and the framework choice. The design review screen (Fig. 4.2) shows the entities, permissions and actions for approval. The workspace (Fig. 4.3) has the chat on the left and, on the right, the live pipeline of agents, the generated files, the tests, the ledger and the running app. While a build runs, the Live view (Fig. 4.4) shows each agent's progress, the activity log that feeds the change ledger, and the files as they are written. After a build, the Quality view lists the bugs the agents found, the tool calls the Reflector made and the fix diffs. The screenshots in this section were captured with the deterministic offline demo model (`FAKE_LLM=1`), which is why the header shows "Offline demo model".

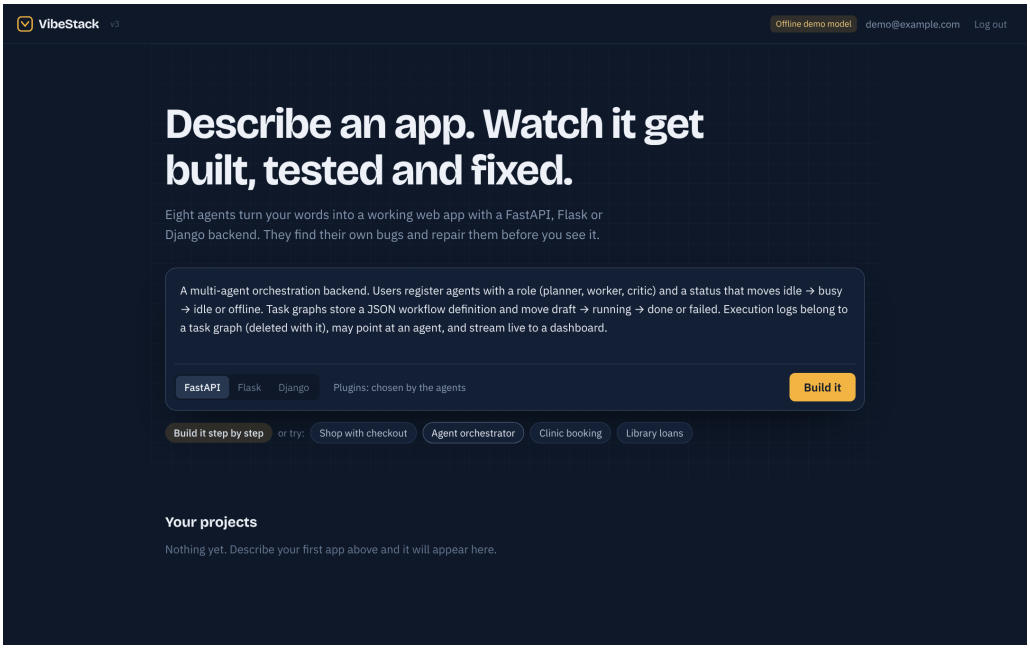


Figure 4.1: Home page with the guided prompt builder (offline demo mode)

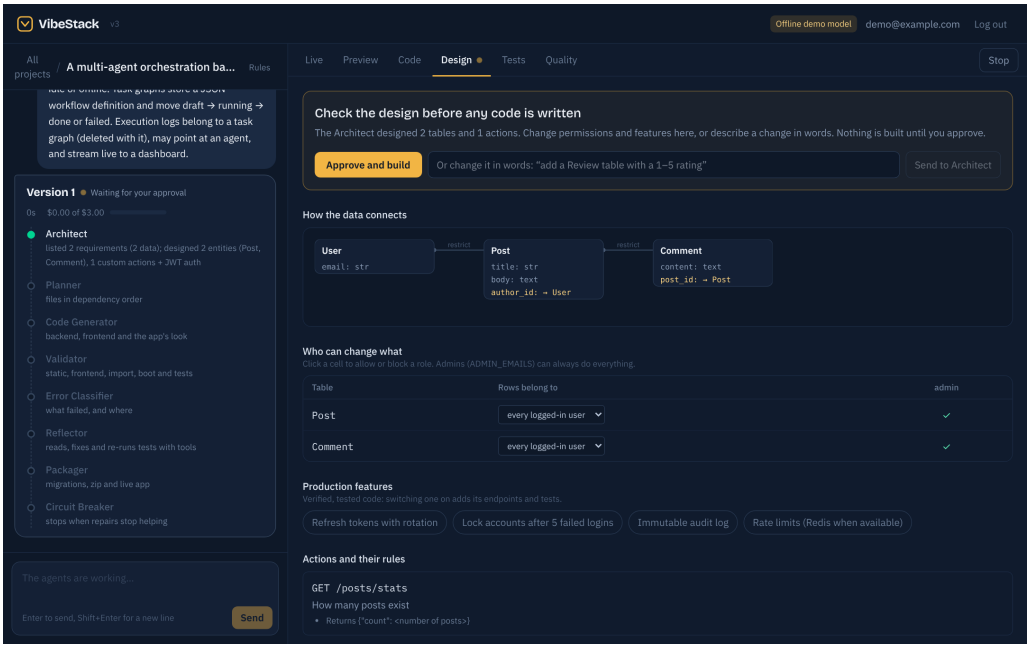


Figure 4.2: Design review before code generation (offline demo mode)

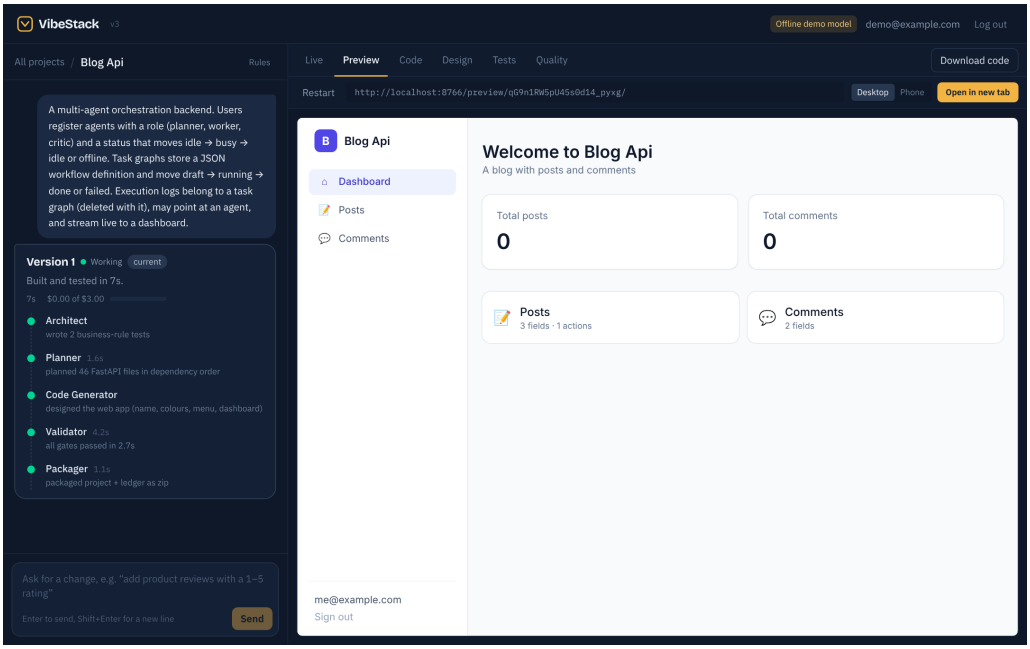


Figure 4.3: Project workspace with the agent pipeline and the live app preview (offline demo mode)

4.8 Testing Approach

Testing an LLM system is tricky because the model's output changes from call to call. We solved this with a deterministic fake model (`agent/fake_llm.py`) that returns fixed, realistic answers. With it, the whole graph, including the sandbox, all three frame-

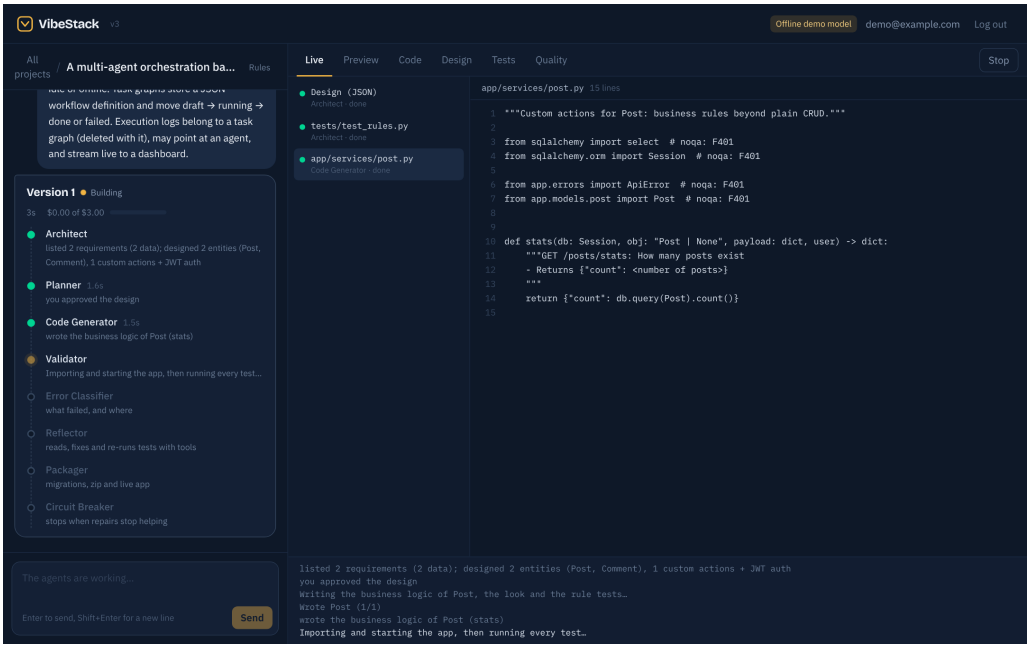


Figure 4.4: Live view during a build: agent progress, activity log and the code being written (offline demo mode)

works and the healing loop, runs in CI without any API key or token cost. Real-model behaviour is checked separately with `scripts/check_llm.py` and with the benchmark runner. GitHub Actions runs the platform tests on a real PostgreSQL database and runs a browser smoke test of the interface.

Chapter 5

Preliminary Results

This chapter reports what has been measured so far. We have been careful to separate results from the automated test suite, which are fully reproducible, from the single real-model measurements taken for Review 1, and from the benchmark evaluation that is still in progress.

5.1 Automated Test Suite

The repository contains about 120 automated tests across 17 test files (Table 5.1). They run with the deterministic fake model and a real sandbox, so every run executes generated FastAPI, Flask and Django applications for real. All of them pass on the main branch. At Review 1 the suite had 36 tests. It has grown as features were added, and each new feature came with its own tests.

5.2 Benchmark Suite

The benchmark (`backend/evals/prompts.yaml`) has 15 prompts. Ten are **standard** applications: todo, blog, library, e-commerce, clinic, event ticketing, fitness tracker, helpdesk, recipes and fleet management. Five are **hard** prompts with business rules, permissions and custom actions: checkout, clinic, library, project management and ticketing. Each prompt also has a reference spec, so the harness can be tested offline without

Table 5.1: Automated tests by area (main branch)

Test file	**What it checks**	**Tests**
`test_healing.py`	Error classes, localisation, progress and stopping rules	23
`test_graph.py`	End-to-end graph runs, healing, follow-ups	13
`test_tools.py`	Reflector tools and tool limits	12
`test_api.py`	Platform HTTP API	9
`test_plugins.py`	Plugin loading and hooks	9
`test_codegen.py`	Template rendering, edge-case tests	8
`test_spec.py`	`ProjectSpec` validation	7
`test_ops.py`	Operations: limits, health, metrics	7
`test_testwriter.py`	Rule tests, calibration, judge	7
`test_deploy.py`	Kubernetes deployment	6
`test_queue.py`	Redis queue and workers	6
`test_bughunt.py`	Static checks	5
`test_robustness.py`	Stops, deadlines, idempotent starts	5
`test_frontend.py`	Frontend contract check	3
`test_projects.py`	Projects and versions	3
`test_evals.py`, `test_storage.py`, `test_mcp.py`	Eval runner, storage, MCP server	5
Total		**121**

a model.

5.3 Interim Measurements (Review 1)

The following numbers were measured for the Review 1 interim evaluation on 8 October 2026, on the version of the system that existed at that time.

The planted-bug evaluation was a deliberate test of the healing loop: a known bug was inserted into each of the ten benchmark projects to check that the loop could find and fix it. Since Review 1, planted bugs have been removed from the system entirely. The agents now have to find real bugs on their own, through the static checks, edge-case tests and rule tests. This is a harder and more honest test, and it is what the Phase II benchmark will measure.

Table 5.2: Interim measurements reported at Review 1

Measurement	**Value**
End-to-end real run on Railway (blog app with JWT auth)	about 51 seconds
API cost of that run	US$0.009 (about 9.6k tokens)
Gates passed in that run	3 of 3 (import, boot, API tests)
Benchmark projects healed (planted-bug harness evaluation)	10 of 10
Repair iterations needed per planted bug	1

5.4 Performance Indicators

Table 5.3 compares the targets we set at the start with what the prototype has achieved so far.

Table 5.3: Expected and achieved performance indicators

Performance indicator	**Expected**	**Achieved (prototype)**
Architecture produced from a prompt	Valid, schema-checked spec	Yes, on every run
Generated app passes all gates	$\geq$ 80% of benchmark prompts	10/10 after healing (Review 1 harness evaluation)
Repair attempts per failure	Bounded, without giving up early	1 on the planted-bug runs; now progress-driven
API cost per generated app	$<$ US$0.25	US$0.009 with budget models, about US$0.20 with a strong model
Frameworks supported	FastAPI	FastAPI, Flask and Django

5.5 Planned Evaluation and Ablation Design

The evaluation runner (`backend/evals/run.py`) is complete and runs end to end offline. For each prompt and framework it reports the pass rate, first-try passes, the number of repairs, the bugs found and fixed, *escaped* bugs, tool calls, tokens, cost and time, and it writes the results as Markdown, JSON and CSV. Escaped bugs are counted on a held-out

run of the edge-case tests, and of any rule tests a configuration did not use, against the final code.

To measure what each idea contributes, the runner compares the full system with four variants:

- **fixed-3:** healing limited to a fixed three repairs;
- **no-tools:** a one-shot Reflector without tools;
- **no-calibration:** rule tests are not checked against the verified templates;
- **no-rule-tests:** the Architect's rule tests are kept out of validation.

Running this benchmark on real models across all three frameworks is the main evaluation task for Phase II. We have deliberately not reported numbers from it here because those runs are not complete.

Chapter 6

Project Schedule

Fig. 6.1 shows the Gantt chart for the project. Table 6.1 lists the milestones reached so far, with dates taken from the review schedule and the project's commit history, and Table 6.2 lists the work that remains.

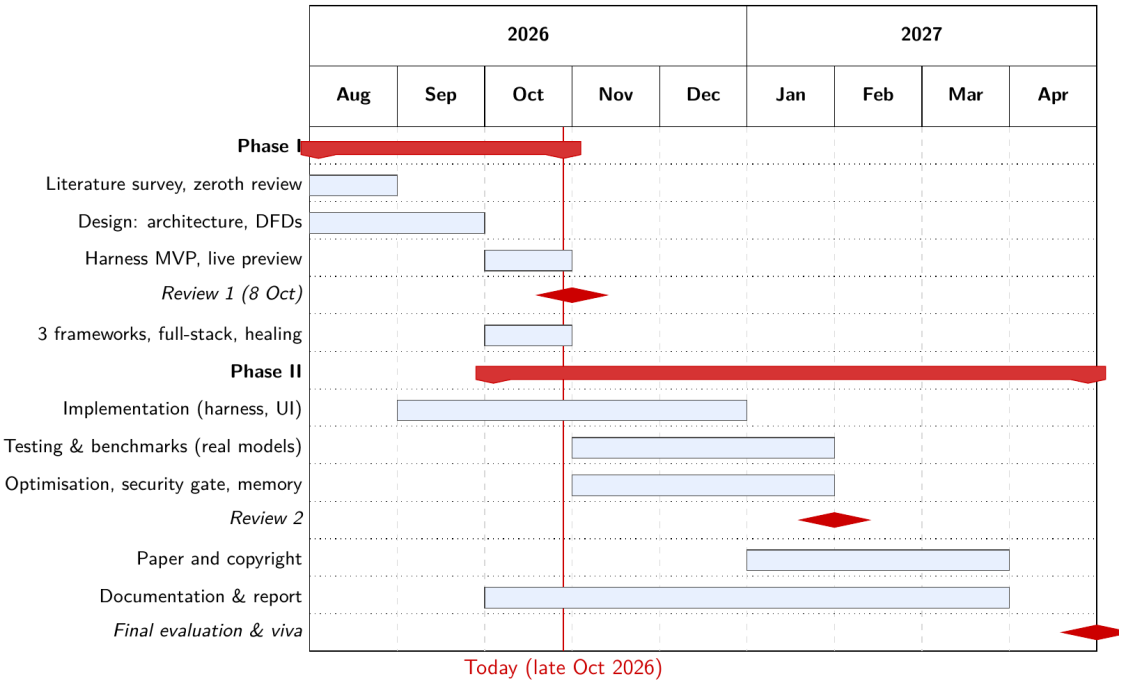


Figure 6.1: Gantt chart of the project

Table 6.1: Milestones completed

Date	**Milestone**
Aug 2026	Literature survey and zeroth review
Aug–Sep 2026	Design: architecture, data flow diagrams, agent roles
5 Oct 2026	First working harness: LangGraph pipeline with self-healing for FastAPI backends
7 Oct 2026	Live preview of generated apps; projector-friendly UI with live pipeline and fix diffs
8 Oct 2026	Review 1 (interim evaluation)
14 Oct 2026	Planted bugs removed; progress-driven healing; Flask and Django support; accounts, quotas, sandbox limits, CI; MCP server and plugins
16 Oct 2026	Full-stack apps with React frontend; projects with follow-up edits; tool-using Reflector; ablation evaluation; provider switch
17 Oct 2026	AI writes only business logic; rule-test calibration; production features as templates; Redis workers; Kubernetes manifests
18 Oct 2026	Design review before code; agents choose plugins; one-click Kubernetes deploy; multi-file repairs; build deadlines; bundled MCP tool servers

Table 6.2: Remaining work and planned timeline

Activity	**Deliverable**	**Planned period**
Testing	Benchmark report: 15 prompts, 3 frameworks, real models, ablations	Nov 2026 – Jan 2027
Optimisation	Lower token cost and latency (prompt caching)	Nov – Dec 2026
Security	Trivy / Checkov scan gate added to the sandbox	Dec 2026
Integration	Long-term memory of past fixes (ChromaDB)	Dec 2026 – Jan 2027
Review 2	Second review	Jan 2027
Publication	Conference paper on the self-healing harness	Jan – Mar 2027
Copyright	Software copyright registration for VibeStack	Feb – Mar 2027
Documentation	Final report, user guide, API documentation	Oct 2026 – Mar 2027
Final	Final project evaluation and viva	Mar – Apr 2027

Chapter 7

Conclusion

In Phase I we set out to build a system that does not just generate backend code but checks and repairs it before handing it over. That system now exists and is merged into the main branch. Eight agents, connected as a LangGraph state machine, take a plain-English description through design, generation, sandboxed validation and repair, and deliver a full-stack application with a FastAPI, Flask or Django backend and a React frontend, together with its tests, migrations, Docker setup and a ledger of every change.

Three ideas have shaped the system most. First, letting the model write only the business logic, while verified templates provide everything else, made builds faster and removed whole classes of errors. Second, deriving edge-case tests from the design without any AI gave us an independent check that the model cannot game, and calibrating the model's own tests stopped wrong tests from steering the repairs. Third, replacing the fixed three-attempt limit with a progress-based stopping rule means that healing continues while it is helping and stops, with a clear reason, when it is not.

7.1 Limitations

- The system targets CRUD-style applications with business rules. Arbitrary algorithms and non-web software are outside its scope.
- The full benchmark on real models has not been completed yet, so the success

rates and costs reported here come from the test suite and from the Review 1 measurements.

- Results depend on the language model provider, and model behaviour can change between versions.
- The edge-case tests serve both as part of validation and as the held-out oracle in the evaluation. The evaluation design accounts for this by reporting held-out numbers only for configurations that did not use them.

7.2 Future Work

In Phase II we will run the full benchmark of fifteen prompts across three frameworks on real models, with the ablation studies, and report the pass rate, escaped bugs, cost and time. We also plan to add a security scanning gate (Trivy and Checkov) to the sandbox, reduce cost and latency through prompt caching, give the Reflector a memory of past fixes, write a conference paper on progress-driven self-healing, and register the software for copyright.

Bibliography

[1] GitHub, "GitHub Copilot documentation," https://docs.github.com/copilot, 2025.

[2] M. Chen, J. Tworek, H. Jun, Q. Yuan *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[3] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[4] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang *et al.*, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[5] M. Jones, J. Bradley, and N. Sakimura, "JSON web token (JWT)," RFC 7519, Internet Engineering Task Force, 2015.

[6] United Nations, "Goal 9: Build resilient infrastructure, promote sustainable industrialization and foster innovation," https://sdgs.un.org/goals/goal9, 2015.

[7] A. Osika *et al.*, "gpt-engineer," https://github.com/gpt-engineer-org/gpt-engineer, 2023.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[9] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.

[10] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *International Conference on Learning Representations (ICLR)*, 2024.

[11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.

[13] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[14] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv preprint arXiv:2312.13010*, 2023.

[15] Vercel, "Next.js documentation," https://nextjs.org/docs, 2025.

[16] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com, 2025.

[17] Anthropic, "Model context protocol specification," https://modelcontextprotocol.io, 2025.

[18] LangChain Inc., "LangGraph documentation," https://langchain-ai.github.io/langgraph/, 2025.

[19] OpenRouter, "OpenRouter API reference," https://openrouter.ai/docs, 2025.

[20] Pydantic, "Pydantic v2 documentation," https://docs.pydantic.dev, 2025.

[21] pytest-dev, "pytest documentation," https://docs.pytest.org, 2025.

[22] M. Nottingham and E. Wilde, "Problem details for HTTP APIs," RFC 7807, Internet Engineering Task Force, 2016.

Appendix A

Turnitin Plagiarism Report

[Insert Plagiarism Report Here]

Appendix B

Base Paper (most relevant one)

[Insert Base Paper Here]